Обработка исключений

Обработка исключений

```
# Commit to create tha table
connection.commit()

try:

    # Insert data into table
    sql = "INSERT INTO Person (NAME, HEIGHT) VALUES ('Jayanam', 345345)"
    cursor.execute(sql)
    connection.commit()

except sqlite3.DatabaseError as e:
    print("Error: %s" % (e.args[0]))
```

```
Run  sqlite_test
C:\Projekte\Python34\python.exe C:/Users/
Error: no such table: Person55

Process finished with exit code 0
```

Python: Exception handling tutorial

```
    cursor.execute(sql)
    rows = cursor.fetchall()

    for row in rows:
        print(row)
except sqlite3.DatabaseError as e:
```

Что такое исключения?

Типы ошибок:

- Синтаксические – возникают из-за синтаксических погрешностей кода;
- Логические – проявляются вследствие логических неточностей в алгоритме;
- Исключения – вызваны некорректными действиями пользователя или системы.

Синтаксические ошибки являются следствием несоблюдения общепринятых норм языка, например, пропуска круглой скобки в предполагаемом месте или некорректного написания имени функции. Такие погрешности успешно отлавливаются компилятором, который сразу же сообщает пользователю о проблеме в написанном коде.

```
  File "main.py", line 1
    print("Hello World!)
                        ^
SyntaxError: EOL while scanning string literal
```

- **Логические ошибки** считаются более сложными в выявлении, поскольку не отлавливаются ни на этапе компиляции, ни во время выполнения готовой программы. Обычно они вызваны определенными недостатками в логике написанного алгоритма, из-за чего пользователь не получает желаемого результата.

```
def avg(a, b):
    return a + b / 2
print(avg(10, 20))


20
```

Исключения представляют собой еще один вид ошибок, который проявляется в зависимости от наличия обстоятельств, меняющих ход выполнения программы. Примером этому вполне может быть ввод некорректного значения либо отсутствие нужного файла. Как правило, все доступные виды исключений становятся видимыми во время выполнения программы. Пример исключения при попытке деления на ноль:

```
Traceback (most recent call last):
File "main.py", line 1, in <module>
print(10 / 0)
ZeroDivisionError: division by zero
```

- В примере деление на ноль провоцирует исключительную ситуацию, которая приводит к аварийному завершению работы и выводу ошибки на экран.
- Для того, чтобы исключение не приводило к внезапному завершению программы, его стоит обработать. Для этого есть специальный механизм, предотвращающего все непредвиденные ситуации.

Некоторые типы исключений

- **AttributeError** - объект не имеет данного атрибута (значения или метода)
- **FileNotFoundError** - файл или директория не существует.
- **IndexError** – генерируется, если в последовательности не найден элемент с заданным индексом.
- **KeyError** – если в словаре не найден указанный ключ.
- **NameError** – если не найдено имя(переменной или функции).
- **SyntaxError** - синтаксическая ошибка.
- **TypeError** - операция применена к объекту несоответствующего типа.
- **ValueError** - функция получает аргумент правильного типа, но некорректного значения.
- **UnicodeError** - ошибка, связанная с кодированием / раскодированием unicode в строках
- **ZeroDivisionError** – если есть деление на 0.

Конструкция try - except

```
try:
    x = 10 / 0  # В этом блоке могут быть ошибки
except ZeroDivisionError:  # обязательно указать тип ошибки
    x = None    # выполняется, если блок try выбрасывает ошибку
else:
    print('Ошибок нет')  # выполняется, если в блоке try нет ошибок
finally:
    print(x)    # Этот блок выполняется всегда
```

Блок try

- Блок **try** это блок кода, который вы хотите попробовать выполнить. Однако во время выполнения из-за какого-нибудь исключения могут возникнуть ошибки. Поэтому этот блок может не работать должным образом.

Блок **except**

- Блок **except** запускается, когда блок **try** не срабатывает из-за исключения. Инструкции в этом блоке часто дают некоторый контекст того, что пошло не так внутри блока **try**.
- Если собираетесь перехватить ошибку как исключение, в блоке **except** нужно обязательно указать тип этой ошибки.
- **except** можно использовать и без указания типа ошибки. Но лучше так не делать. При таком подходе не учитывается, что возникающие ошибки могут быть разных типов. То есть вы будете знать, что что-то пошло не так, но что именно произошло, какая была ошибка — вам будет не известно.

As — сохраняет ошибку в переменную

- Перехваченная ошибка представляет собой объект класса, унаследованного от "BaseException". С помощью ключевого слова as можно записать этот объект в переменную, чтобы обратиться к нему внутри блока except.

```
try:
    file = open('text_file.txt', 'r')
except FileNotFoundError as e:
    print(e)
```

```
> >> [Errno 2] No such file or directory: 'text_file.txt.txt
```

Несколько блоков **except**

- При попытке выполнить код внутри блока try также существует вероятность возникновения нескольких ошибок.
- Например, вы можете попытаться обратиться к элементу списка по индексу, выходящему за пределы допустимого диапазона, использовать неправильный ключ словаря и попробовать открыть несуществующий файл – и все это внутри одного блока try.
- В результате вы можете столкнуться с IndexError, KeyError и FileNotFoundError. В таком случае нужно добавить столько блоков except, сколько ошибок ожидается – по одному для каждого типа ошибки.

Несколько блоков **except**

```
try:
    print("Opening file...")
    f = open("data.txt")
except FileNotFoundError:
    print("файл не найден!")
except Exception:
    print("все остальные ошибки, кроме системных")
print("Program finished")
```

Несколько типов исключений в одном блоке **except**

- Можно также обрабатывать в одном блоке except сразу несколько исключений. Для этого они записываются в круглых скобках, через запятую сразу после ключевого слова except:

```
try:
    b = float(input('b = '))
    c = float(input('c = '))
    a = b / c
except (ZeroDivisionError, ValueError) as er:
    print(er)
else:
    print('a = ', a)
```

Блок else

- Блок else запускается только в том случае, если блок try выполняется без ошибок. Это может быть полезно, когда нужно выполнить ещё какие-то действия после успешного выполнения блока try. Например, после успешного открытия файла вы можете прочитать его содержимое.

Блок finally

- Блок finally выполняется всегда, независимо от того, что происходит в других блоках. Это полезно, когда вы хотите освободить ресурсы после выполнения определенного блока кода.

- Примечание: блоки else и finally не являются обязательными.

Raise — самостоятельный вызов исключений

- Исключения можно генерировать самостоятельно — для этого нужно запустить оператор **raise**.

```
age = float(input('введите возраст: '))
if age < 0:
    raise Exception('Возраст должен быть больше нуля!')
```

Перехватываются такие сообщения точно так же, как и остальные:

```
try:
    age = float(input('введите возраст: '))
    if age < 0:
        raise Exception('Возраст должен быть больше нуля!')
except Exception:
    print("Ошибка пользователя!")
```

```
>>> введите возраст: -10
>>> Ошибка пользователя!
```

Иерархия встроенных в python исключений

- **BaseException** - базовое исключение, от которого берут начало все остальные.
- **SystemExit** - исключение, порождаемое функцией sys.exit при выходе из программы.
- **KeyboardInterrupt** - порождается при прерывании программы пользователем (обычно сочетанием клавиш Ctrl+C).
- **GeneratorExit** - порождается при вызове метода close объекта generator.
- **Exception** - а вот тут уже заканчиваются полностью системные исключения (которые лучше не трогать) и начинаются обыкновенные, с которыми можно работать.

Как создать **свой** тип Exception

- В Python можно создавать свои исключения. При этом есть одно обязательное условие: они должны быть потомками класса **Exception**:

```
class MyError(Exception):
    def __init__(self, text):
        self.txt = text

try:
    raise MyError('Моя ошибка')
except MyError as er:
    print(er)
```

Задание для самостоятельного решения.

1. Программа высчитывает квадратный корень числа.

Модифицируйте код так, что бы не происходило аварийное завершение, если пользователь введет отрицательное число или строку.

```
from math import sqrt
x = input('Введите число: ')
print('Корень числа:', sqrt(x))
```

2. Программа высчитывает отношение чисел.
Модифицируйте код так, что бы не происходило аварийное завершение, если пользователь введет только одно число или ноль в качестве делителя.

```
def division():
    nums = input('Введите два числа через пробел: ').split()
    return float(nums[0]) / float(nums[1])

division()
```